Warp divergence

GPU Programming – M2 DiPaC, Université Paris-Saclay – Session 4

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Objectives

- Understand what happens when the 32 threads of a warp do not agree on a branch: **divergence**, and what it costs
- See what the hardware does with a branch: masks, predication, select, reconvergence
- Recognise the branches that diverge, the ones that do not, and the ones that do not matter
- See that loops diverge too, and how the order of the data decides how much
- Know the remedies: align the condition on warps, reorder the data, replace short branches by arithmetic
- Measure all of it on the lab GPU

Outline

1. Recap of session 3
2. Warp divergence
3. Summary

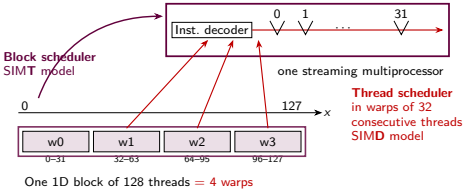
Outline

1. Recap of session 3
2. Warp divergence
3. Summary

Recap: warps, the unit of execution

Make blocks out of a whole number of **warps**

- One instruction decoder drives 32 CUDA cores, i.e. 32 threads at once
- Each group of 32 consecutive threads of a block is a **warp**: the unit the hardware actually schedules
- The threads of a warp are **synchronous**: they execute the same instruction at the same time, each on its own data
- Threads are numbered `threadIdx.x` first, then `.y`, then `.z`; that order is cut into warps of 32
- **Today's question**: what if the 32 threads do not want to execute the *same* instruction?



Outline

1. Recap of session 3
2. Warp divergence
3. Summary

Branching in a GPU kernel

The 32 threads of a warp execute **one instruction at a time, together**. What about `if (cond) f(); else g();`?

- If **all** threads of the warp satisfy `cond`: the warp executes `f()` only. Time F
- If **none** does: `g()` only. Time G
- If **at least one** satisfies `cond` and **at least one** fails it, the warp **diverges**:
  - `f()` is executed first, with the threads that fail `cond` **masked off** (idle threads)
  - then `g()`, with the threads that satisfy `cond` masked off
  - the two paths are **serialised**: time $F + G$, whatever the proportion of threads on each side
- The result is correct – every thread executes its own path – but the masked threads are wasted
- Divergence is decided **warp by warp**: never block by block, never thread by thread

```
// A branch whose condition depends on the data:
// each thread goes its own way
__global__ void branch(float *x)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (x[i] > 0.0f) { x[i] = f(x[i]); }
  else             { x[i] = g(x[i]); }
```

Here `cond` depends on the data: which threads of a warp take which side is only known when the warp gets there

Example: two kinds of branch

- `f()` and `g()`: two loops of multiply-adds, of times F and G
- Kernel 1: condition on the parity of the index `i`
- Kernel 2: condition on the parity of the **warp number** `i / 32`
- **Which kernels diverge? What is the time of each warp?**

```
// (1) parity of i: every warp has both kinds
__global__ void divergeEveryWarp(int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) {
    if (i % 2 == 0) { dX[i] = f(dX[i]); }
    else            { dX[i] = g(dX[i]); }
  }
}

// (2) parity of the warp number i / 32
__global__ void divergeNoWarp(int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) {
    if ((i / 32) % 2 == 0) { dX[i] = f(dX[i]); }
    else                   { dX[i] = g(dX[i]); }
  }
}
```

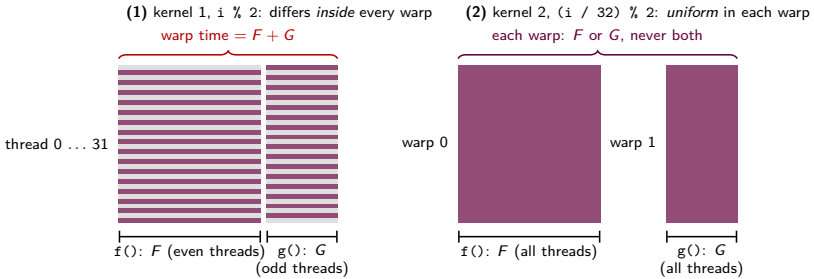
Example: two kinds of branch

- `f()` and `g()`: two loops of multiply-adds, of times F and G
- Kernel 1: condition on the parity of the index `i`
- Kernel 2: condition on the parity of the **warp number** `i / 32`
- **Which kernels diverge? What is the time of each warp?**
- Kernel 1 diverges in every warp: `f()` then `g()`, time $F + G$
- Kernel 2: a warp is entirely `f` or entirely `g`: time F or G, never both

```
// (1) parity of i: every warp has both kinds
__global__ void divergeEveryWarp(int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) {
    if (i % 2 == 0) { dX[i] = f(dX[i]); }
    else            { dX[i] = g(dX[i]); }
  }
}

// (2) parity of the warp number i / 32
__global__ void divergeNoWarp(int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) {
    if ((i / 32) % 2 == 0) { dX[i] = f(dX[i]); }
    else                   { dX[i] = g(dX[i]); }
  }
}
```

Divergence, thread by thread



purple: thread executing; grey: thread idle (masked off); time runs left to right

- Kernel 1: 16 threads on each side in every warp; each warp runs `f()` then `g()`, half the threads idle at every step: $F + G$
- Kernel 2: the condition depends on the warp number `i / 32`: **uniform** within each warp, no thread ever idles: F or G

What the hardware does with a branch

- One instruction stream per warp, plus a 32-bit **active mask**: a thread whose bit is 0 executes the instruction *with no effect* – no register write, no memory request
- `cond ? a : b`, `min`, `max`, `abs`: a **select** instruction (`selp`): both values computed, one kept – **no branch at all**
- A short `if/else`: the assembler **predicates** it (`@P0 FFMA`, `@!P0 FFMA`): every thread walks both sides under its mask bit; cost = both sides, no jump
- A long one (loops, calls): a real **branch**. Uniform predicate: the warp jumps as one. Otherwise a **reconvergence point** is recorded, one side runs under the mask, then the other, then they reconverge: divergence, $F + G$
- Since Volta (2017), **independent thread scheduling**: a program counter per thread, the two sides may be *interleaved*. Same cost model, but **no lockstep guarantee** inside a divergent region: `__syncwarp()` before relying on what other threads wrote

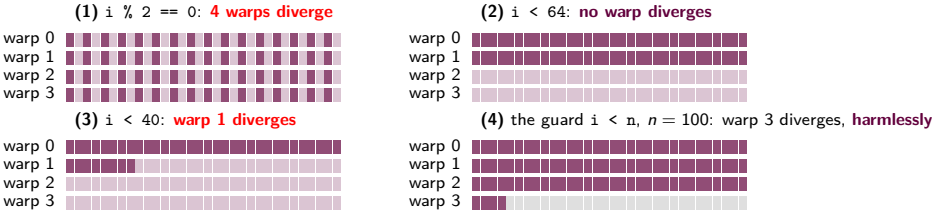
```
// y[i] = (x[i] > 0) ? 2*x[i] : 3*x[i];
  ld.global.f32  %f1, [%rd5];
  setp.gt.f32    %p2, %f1, 0.0;  // p2 = x>0
  selp.f32       %f2, 2.0, 3.0, %p2; // select
  mul.f32        %f3, %f1, %f2;
  st.global.f32  [%rd7], %f3;    // no branch

// if (x[i] > 0) y[i] = 2*x+1;
// else          y[i] = 3*x-1;
  ld.global.f32  %f1, [%rd6];
  setp.gt.f32    %p2, %f1, 0.0;
  @%p2 bra       L_then;         // branch
  fma.rn.f32     %f2, %f1, 3.0, -1.0;
  st.global.f32  [%rd1], %f2;    // else side
  bra.uni        L_end;
L_then:
  fma.rn.f32     %f3, %f1, 2.0, 1.0;
  st.global.f32  [%rd1], %f3;    // then side
L_end:
  ret;
```

PTX from `cuda.godbolt.org` (`-O2 -arch=sm_75`, trimmed); the assembler then predicates the second one

Which warps diverge?

A block of 128 threads = 4 warps (the first block, `i = threadIdx.x`); the two colours are the two sides of the branch

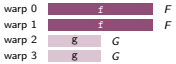


dark: threads taking the `if` branch; light: the `else` branch; grey: nothing to do

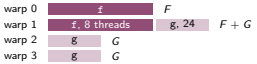
- Only the warps in which *both* colours appear pay $F + G$; a boundary that falls on a multiple of 32 costs nothing

Three cases of `i < k`

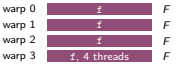
(a) `branchK`, $k = 64$: cut on a warp boundary
no warp diverges



(b) `branchK`, $k = 40$: cut inside warp 1
warp 1 diverges



(c) `guardOnly`, $n = 100$: the guard, no `else`
warp 3 diverges, **nothing serialised**



dark: `f()` running; light: `g()` running; time runs left to right; block of 128 threads, first block

- In `branchK`, `i < k` is decided warp by warp: warps entirely below k run `f()`, warps entirely above run `g()`; only the warp that *contains* k runs both
- (a) `branchK`, $k = 64$: the cut is on a warp boundary; every warp takes F or G, the block $\max(F, G)$: **no divergence**
- (b) `branchK`, $k = 40$: warp 1 runs `f()` with 8 threads, then `g()` with 24: $F + G$; the other warps are unaffected, but the block waits for warp 1
- (c) `guardOnly`, $n = 100$ – the guard `if (i < n)` with no `else`: warp 3 runs `f()` with 4 threads, then nothing: F. Every kernel with a guard diverges in its last warp, **at no cost**: completely normal

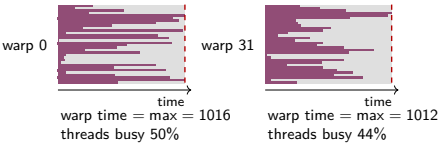
```
// A branch on the index: the threads below k
// do f, the others do g
__global__ void branchK(int n, int k, float *x)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) {
    if (i < k) { x[i] = f(x[i]); }
    else       { x[i] = g(x[i]); }
  }

  ...

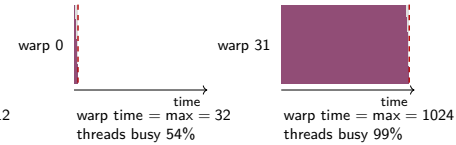
// The guard alone: a branch with an empty else
__global__ void guardOnly(int n, float *x)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) { x[i] = f(x[i]); }
```

Loops diverge too

random: every warp looks alike



sorted: warp w holds the values of rank $32w \dots 32w + 31$, increasing with w



- A loop is a branch taken repeatedly: `for (k = 0; k < iters[i]; k++)`, a different `iters[i]` per thread. The warp runs it **as many times as its longest thread**, the others masked off once done: thread utilisation = mean/ max of the counts in the warp
- Random counts in $1 \dots 1024$: max ≈ 1000 in *every* warp, mean ≈ 512: **half the threads idle**
- **Sorted**: the 32 counts of a warp are close together, max $\approx$ mean. Warp 0 takes ≈ 32 iterations, warp 31 ≈ 1024: over all the warps, **half the time** for the same total work
- **Rule:** put threads with similar amounts of work in the same warp – sort, or reorder the data

```
__global__ void variableWork(int n,
    const int *iters, float *x)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) {
    float v = x[i];
    for (int k = 0; k < iters[i]; k++) {
      v = v * 0.999f + 0.5f;
    }
    x[i] = v;
  }
}
```

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no else
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no else
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no else
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case
- (b) **Every warp**: 16 threads each side, $F + G$ everywhere

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no else
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case
- (b) **Every warp**: 16 threads each side, $F + G$ everywhere
- (c) **Never**: uniform over the block, hence over every warp

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no else
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case
- (b) **Every warp**: 16 threads each side, $F + G$ everywhere
- (c) **Never**: uniform over the block, hence over every warp
- (d) Warp 0 of each block diverges – one store, no `else`: **harmless**

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no `else`
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case
- (b) **Every warp**: 16 threads each side, $F + G$ everywhere
- (c) **Never**: uniform over the block, hence over every warp
- (d) Warp 0 of each block diverges – one store, no `else`: **harmless**
- (e) **Never** – if `blockDim.x` is a multiple of 32; otherwise some warps contain threads from both sides

Exercise: 1D kernels

Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no `else`
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case
- (b) **Every warp**: 16 threads each side, $F + G$ everywhere
- (c) **Never**: uniform over the block, hence over every warp
- (d) Warp 0 of each block diverges – one store, no `else`: **harmless**
- (e) **Never** – if `blockDim.x` is a multiple of 32; otherwise some warps contain threads from both sides
- (f) Trip counts $i_0 \dots i_0 + 31$ in a warp: at most 31 idle iterations per thread, **negligible** once $i \gg 32$

Exercise: 1D kernels

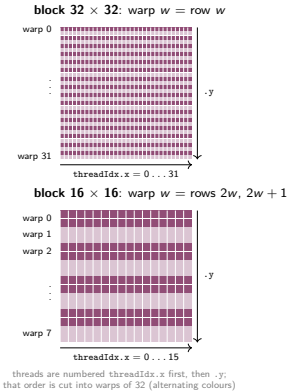
Blocks of 256 threads, `n` large, `f()` and `g()` long. Which warps diverge?

```
// The kernel of the exercise: COND is one of the
// conditions (a) to (g)
__global__ void kernel(const float *x, float *y,
                       const int *type, float *s)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (COND) { y[i] = f(x[i]); }
  else      { y[i] = g(x[i]); }
}
```

- (a) `x[i] > 0`
- (b) `i % 2 == 0`
- (c) `blockIdx.x % 2 == 0`
- (d) `threadIdx.x == 0`, body `s[blockIdx.x] = v;`, no `else`
- (e) `(i / 32) % 2 == 0`
- (f) instead of the `if`: `for (k = 0; k < i; k++) { ...}`
- (g) instead of the `if`: `switch (type[i])`, four cases

- (a) **Data-dependent**: every warp holding both signs pays $F + G$; nothing to know in advance. The usual case
- (b) **Every warp**: 16 threads each side, $F + G$ everywhere
- (c) **Never**: uniform over the block, hence over every warp
- (d) Warp 0 of each block diverges – one store, no `else`: **harmless**
- (e) **Never** – if `blockDim.x` is a multiple of 32; otherwise some warps contain threads from both sides
- (f) Trip counts $i_0 \dots i_0 + 31$ in a warp: at most 31 idle iterations per thread, **negligible** once $i \gg 32$
- (g) A warp runs **every case present** among its 32 threads, up to $4\times$ one case; group or sort by `type`

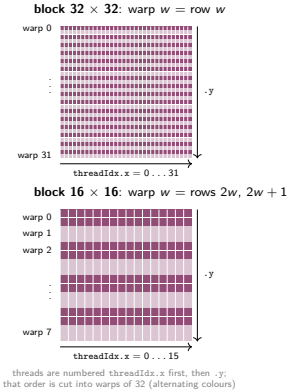
Exercise: 2D kernels



The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

Exercise: 2D kernels



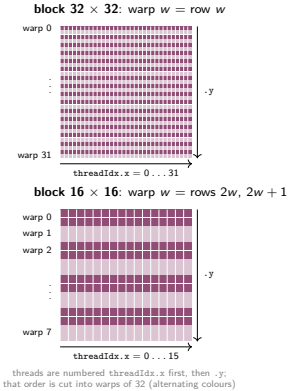
threads are numbered `threadIdx.x` first, then `.y`;
that order is cut into warps of 32 (alternating colours)

The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

- (a) **Never**: a warp is one row, `threadIdx.y` is uniform in it

Exercise: 2D kernels

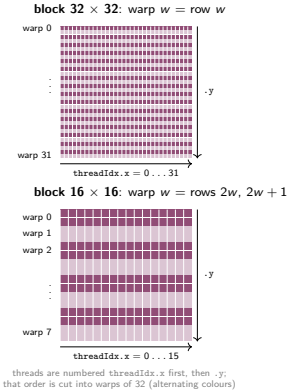


The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

- (a) **Never**: a warp is one row, `threadIdx.y` is uniform in it
- (b) **All 32 warps**: thread 0 against the 31 others

Exercise: 2D kernels

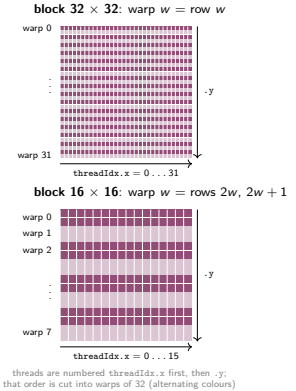


The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

- (a) **Never**: a warp is one row, `threadIdx.y` is uniform in it
- (b) **All 32 warps**: thread 0 against the 31 others
- (c) **All 32 warps**, half and half

Exercise: 2D kernels

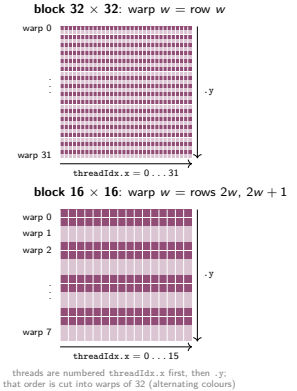


The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

- (a) **Never**: a warp is one row, `threadIdx.y` is uniform in it
- (b) **All 32 warps**: thread 0 against the 31 others
- (c) **All 32 warps**, half and half
- (d) **Every warp**: a warp is rows $2w$ and $2w + 1$, one of each parity – same code, other block shape

Exercise: 2D kernels

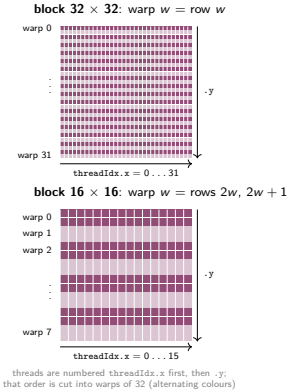


The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

- (a) **Never**: a warp is one row, `threadIdx.y` is uniform in it
- (b) **All 32 warps**: thread 0 against the 31 others
- (c) **All 32 warps**, half and half
- (d) **Every warp**: a warp is rows $2w$ and $2w + 1$, one of each parity – same code, other block shape
- (e) **Never**: warps 0–3 entirely below, 4–7 entirely above

Exercise: 2D kernels



The same kernel with `i`, `j` from a 2D block: `if (COND) f(); else g();`, both long. Which warps diverge?

- (a) block 32×32: `threadIdx.y == 0`
- (b) block 32×32: `threadIdx.x == 0`
- (c) block 32×32: `threadIdx.x < 16`
- (d) block 16×16: `threadIdx.y % 2 == 0`
- (e) block 16×16: `threadIdx.y < 8`
- (f) block 32×32: `i < n && j < n`, no `else`

- (a) **Never**: a warp is one row, `threadIdx.y` is uniform in it
- (b) **All 32 warps**: thread 0 against the 31 others
- (c) **All 32 warps**, half and half
- (d) **Every warp**: a warp is rows $2w$ and $2w + 1$, one of each parity – same code, other block shape
- (e) **Never**: warps 0–3 entirely below, 4–7 entirely above
- (f) The warps that cross the edge of the matrix **diverge** (the threads outside are masked off) – but **harmless**: nothing on the other side

Living with branches

- **Align the condition on warps**: when the two sides are two kinds of work, give each kind to whole warps – `i / 32`, not `i % 2`
- **Reorder the data** so that threads with the same path are contiguous: sort by the branch condition, or by the amount of work
- **Replace the branch by arithmetic** when both sides are cheap: `x = cond ? a : b` compiles to a *select* instruction, no divergence; likewise `max`, `min`, `abs`, masks
- **Do not fight the guard**: `if (i < n)` diverges in one warp with an empty `else`; leave it
- **Do not fight small branches**: a divergent `if` around two instructions costs two instructions; the hardware predicates short branches anyway
- What costs is a **long** divergent region – a loop, a function call – in **many** warps

Divergence: measured

`src/divergence-two-kinds.cu` and `src/divergence-loop.cu`: 2^{22} threads in blocks of 256, Tesla T4 (40 SMs, `cuda.godbolt.org`)

kernel	per warp	time
`uniformF`	F, all threads busy	3.10 ms
`uniformG`	G, all threads busy	1.55 ms
`divergeNoWarp`	F or G, all threads busy	2.32 ms
`divergeEveryWarp`	$F + G$, half the threads idle	4.60 ms
`variableWork`, random	max of 32 random trip counts	2.29 ms
`variableWork`, sorted	max $\approx$ mean	1.20 ms

- `divergeNoWarp`: half the warps do F, half do G, and the 40 SMs run them concurrently – the kernel takes the **average** work, $(F + G)/2$, not the maximum
- `divergeEveryWarp`: every warp does F *then* G: $F + G$, **twice** `divergeNoWarp` for exactly the same instructions
- Sorted $\approx 0.5\times$ random, again for exactly the same instructions
- Divergence is invisible in the result and in the instruction count; only the warp view shows it

Outline

1. Recap of session 3
2. Warp divergence
3. Summary

What to remember

- A warp executes one instruction at a time; a branch on which its threads disagree is **serialised**: both paths, threads masked off, time $F + G$. Divergence is decided warp by warp
- It costs only when both paths have work, and matters when the paths are long and many warps diverge. The guard `if (i < n)` is harmless: its `else` is empty
- Loops with per-thread trip counts run at the warp's maximum: the threads that finish early wait, masked off
- Remedies: align conditions on warps (`i / 32`, not `i % 2`), reorder or sort the data so that similar work lands in the same warp, replace short branches by arithmetic
- Divergence is invisible in the result and in the instruction count; only the warp view shows it – measure

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Gif-sur-Yvette, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com

Course material: `github.com/kayaogz/2627-m2dipac-gpuprogramming`